



Prepared by:
ENG: Haitham Abu Draz

Supervised by:
Masar

Theory Questions :

1) Why is it considered incorrect HTML to place a `<td>` element directly inside a `<table>` without a `<tr>` element? Explain what the browser does in this situation.

Answer:

Why it is incorrect: The HTML specification requires that `<td>` elements must be children of `<tr>` (which itself must be inside `<table>` or a sectioning element like `<tbody>`). Placing a `<td>` directly under `<table>` violates the document structure.

Browser behavior: It automatically creates an implicit `<tr>` element (and often a `<tbody>`) in the Document Object Model (DOM) to wrap the cell, forcing it into a valid structural layout.

2) What is the functional difference between using `<thead>`, `<tbody>`, and `<tfoot>` versus placing all rows directly inside `<table>`?

Answer:

1. `<thead>` (Table Head)

- **Purpose:** Defines the header rows of the table.
- **Content:** Contains labels that describe what each column represents (e.g., "Product Name", "Price").
- **Behavior:** Repeats at the top of every printed page if the table spans multiple pages.

2. `<tbody>` (Table Body)

- **Purpose:** Encapsulates the main data rows of the table.
- **Content:** Contains the actual, functional information and records.
- **Behavior:** You can use multiple `<tbody>` elements in one table to split data into distinct visual sections or categories.

3. `<tfoot>` (Table Foot)

- **Purpose:** Defines the summary or conclusion row(s) of the table.
- **Content:** Contains calculated totals, averages, or overall balances of the columns above.
- **Behavior:** Repeats at the very bottom of every printed page if the table splits across physical pages.

Without sections	With sections
All rows are treated as a single, flat group.	Rows are logically separated into header, body, and footer.
No semantic meaning for screen readers or SEO.	Screen readers can announce headers and repeat them across pages.
The header cannot be repeated on each printed page.	Browsers can repeat <thead> and <tfoot> on every printed page.
Scrolling the body also scrolls the header (no sticky header).	You can make <tbody> scrollable while <thead> stays fixed (with CSS).
No clear grouping for styling or JavaScript.	You can style or manipulate each section independently.

3) Why can the <tfoot> element appear before <tbody> in the HTML source but still be displayed at the bottom of the table?

Answer:

The HTML specification explicitly allows <tfoot> to be placed before <tbody> (or after it) for performance reasons – the footer can be loaded early while the large body data streams in. Browsers render tables in display order: header (top), body (middle), footer (bottom), completely ignoring the source order. This is a deliberate feature of the table rendering model.

4) What would happen if the total number of columns in one row becomes different from the other rows because of incorrect use of colspan or rowspan?

Answer:

The visual layout of the table will break.

- If a row calculates to more columns than the table's defined width, the browser will stretch the table grid, causing irregular cells to protrude out on the right side.
- If a row calculates to fewer columns, empty gaps will appear under or beside cells, misaligning subsequent column headers with their respective data.

5) Why is an empty table cell still written as <td></td> instead of simply omitting the cell?

Answer:

Because tables are built on a grid model – each row must have the same number of columns. Omitting a cell would shift all subsequent cells to the left, breaking alignment with other rows. An empty cell (<td></td>) explicitly reserves that column slot, preserving the grid's integrity.

6) Can a <th> element be placed anywhere inside a table? Explain the correct usage

Answer:

No. A <th> (table header) element must always be placed inside a table row (<tr>). Correct usage dictates that it should represent header data rather than regular layout cells. It can be used inside a <thead> for column headers, or at the beginning of a row inside a <tbody> to serve as a row header.

7) Why is using tables for webpage layout considered poor practice in modern web development?

- **Accessibility:** Screen readers interpret tables as data, confusing visually impaired users.
- **Responsiveness:** Tables are notoriously hard to make responsive; they break on small screens.
- **Maintainability:** Layout tables mix presentation with structure, making code bloated and hard to update.
- **Performance:** Browsers render tables slower than modern layout methods (CSS Grid/Flexbox).
- **Semantic HTML:** We have dedicated layout tools (display: grid, display: flex) that convey structure correctly.

8) If two cells each have colspan="2" inside a table with four columns, how many cells are actually represented in that row? Explain.

Answer:

Exactly two physical cells are written in the HTML code. However, because each cell spans two column tracks, they occupy a total structural space of four columns ($2 + 2 = 4$), perfectly filling the row.

9) Why is it important for every row in a table to represent the same total number of columns, even when using colspan and rowspan?

Answer:

A table relies on a predictable mathematical grid. If rows do not equal the same total column count, the grid breaks, distorting cell borders, ruining alignments, and creating unstyled rendering bugs across different web browsers.

10) Why do browsers usually display HTML tables correctly even when the markup contains mistakes? Is this behavior guaranteed?

Answer:

Browsers have built-in error recovery algorithms (e.g., HTML5 parsing rules) that fix common issues like missing tags.

This behavior is NOT guaranteed – while modern browsers mostly converge on similar fixes, different rendering engines (Blink, Gecko, WebKit) could theoretically handle edge cases differently. Relying on error recovery is risky and invalidates your HTML.

11) What is the main purpose of the "action" attribute in an HTML <form>element?

Answer:

The action attribute specifies the URL (endpoint) where the form data should be sent when the form is submitted. It tells the browser where to make the HTTP request.

12) When should a developer prefer the "post" method over the "get" method for submitting a form?

Answer:

Use post when:

- The data is sensitive (passwords, credit cards, personal info).
- The data is large (file uploads, long text) – get has URL length limits (~2KB).
- The submission changes server state (e.g., updating a profile, deleting a record, placing an order).

get is suitable for idempotent, bookmarkable actions like search queries

13) What is the function of the "name" attribute in form controls like text inputs or checkboxes?

Answer:

The name attribute acts as the key in the key=value pair that is sent to the server when the form is submitted. Without a name, the control's value will not be submitted.

14) What is the difference between a "checkbox" and a "radio button" in terms of user selection?

Answer:

Checkbox: Allows the user to select zero, one, or multiple independent options (each checkbox toggles on/off).

Radio Button: Forces a mutually exclusive selection. When multiple radio buttons share the exact same name attribute, the user can only select a single option from that group.

15) Why is the <textarea>element different from other standard <input>elements?

Answer:

- It is a multi-line text input, whereas <input type="text"> is single-line.
- It requires a closing tag (</textarea>) – its content (default text) goes between the tags, not in a value attribute.
- It supports rows and cols attributes to define its dimensions.

Practical Questions:

1) Create a table with 5 columns total where:

- The first row has a single cell that spans all 5 columns (`colspan="5"`)
- The second row has one cell with `rowspan="3"`
- The remaining structure must stay perfectly aligned for 3 more rows
- No row is allowed to break the column structure

```
<body>
  <table border="1" style="text-align: center;">
    <tr>
      <td colspan="5">Row 1 - Full Width (Col 1-5)</td>
    </tr>

    <tr>
      <td rowspan="3">Row 2,3,4 (Col 1)</td>
      <td>Row 2 (Col 2)</td>
      <td>Row 2 (Col 3)</td>
      <td>Row 2 (Col 4)</td>
      <td>Row 2 (Col 5)</td>
    </tr>

    <tr>
      <td>Row 3 (Col 2)</td>
      <td>Row 3 (Col 3)</td>
      <td>Row 3 (Col 4)</td>
      <td>Row 3 (Col 5)</td>
    </tr>

    <tr>
      <td>Row 4 (Col 2)</td>
      <td>Row 4 (Col 3)</td>
      <td>Row 4 (Col 4)</td>
      <td>Row 4 (Col 5)</td>
    </tr>
  </table>
```

2) Build a table where:

- The first row has 3 columns
- The second row has a cell that spans both 2 columns and 2 rows (combined rowspan + colspan)
- The remaining cells must adjust correctly without creating extra or missing columns
- The table must remain valid HTML structure

```
<table border="1" style="text-align: center;">
  <tr>
    <td>Row1 - Col1</td>
    <td>Row1 - Col2</td>
    <td>Row1 - Col3</td>
  </tr>

  <tr>
    <td rowspan="2" colspan="2">
      Spans 2 cols & 2 rows
    </td>
    <td>Row2 - Col3</td>
  </tr>

  <tr>
    <td>Row3 - Col3</td>
  </tr>
</table>
```

3) Create an HTML form for a user profile page that includes the following requirements:

1. **Form Structure:** The form must send data to a file named `update_profile.php` and use the most secure method for submitting sensitive information.
2. **Password Field:** Include a password input field named `user_password` that allows a maximum of 15 characters, ensuring the input is hidden from view.
3. **Programming Interests:** Create a section labeled "Favorite Languages" using the appropriate input type that allows the user to select multiple options (e.g., HTML, CSS, JavaScript), and ensure the server receives these selections as a group.
4. **Submission:** Add a submit button to the form.

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Task3</title>
</head>

<body>
  <h2>Update Profile</h2>

  <!--
    1) action="http://www.example.com/update_profile"
    2) method="POST" (most secure for sensitive data)
  -->

  <form action="http://www.example.com/update_profile/" method="POST">

    <div>
      <label for="user_password">Password:</label>
      <input type="password" id="user_password" name="user_password" maxlength="15" required>
    </div>

    <br>

    <fieldset>
      <legend>Favorite Languages</legend>

      <label>
        <input type="checkbox" name="languages[]" value="HTML"> HTML
      </label><br>

      <label>
        <input type="checkbox" name="languages[]" value="CSS"> CSS
      </label><br>

      <label>
        <input type="checkbox" name="languages[]" value="JavaScript"> JavaScript
      </label><br>

      <label>
        <input type="checkbox" name="languages[]" value="React"> React
      </label>
    </fieldset>

    <br>

    <div>
      <button type="submit" name="update">Update Profile</button>
    </div>

  </form>
</body>

</html>
```